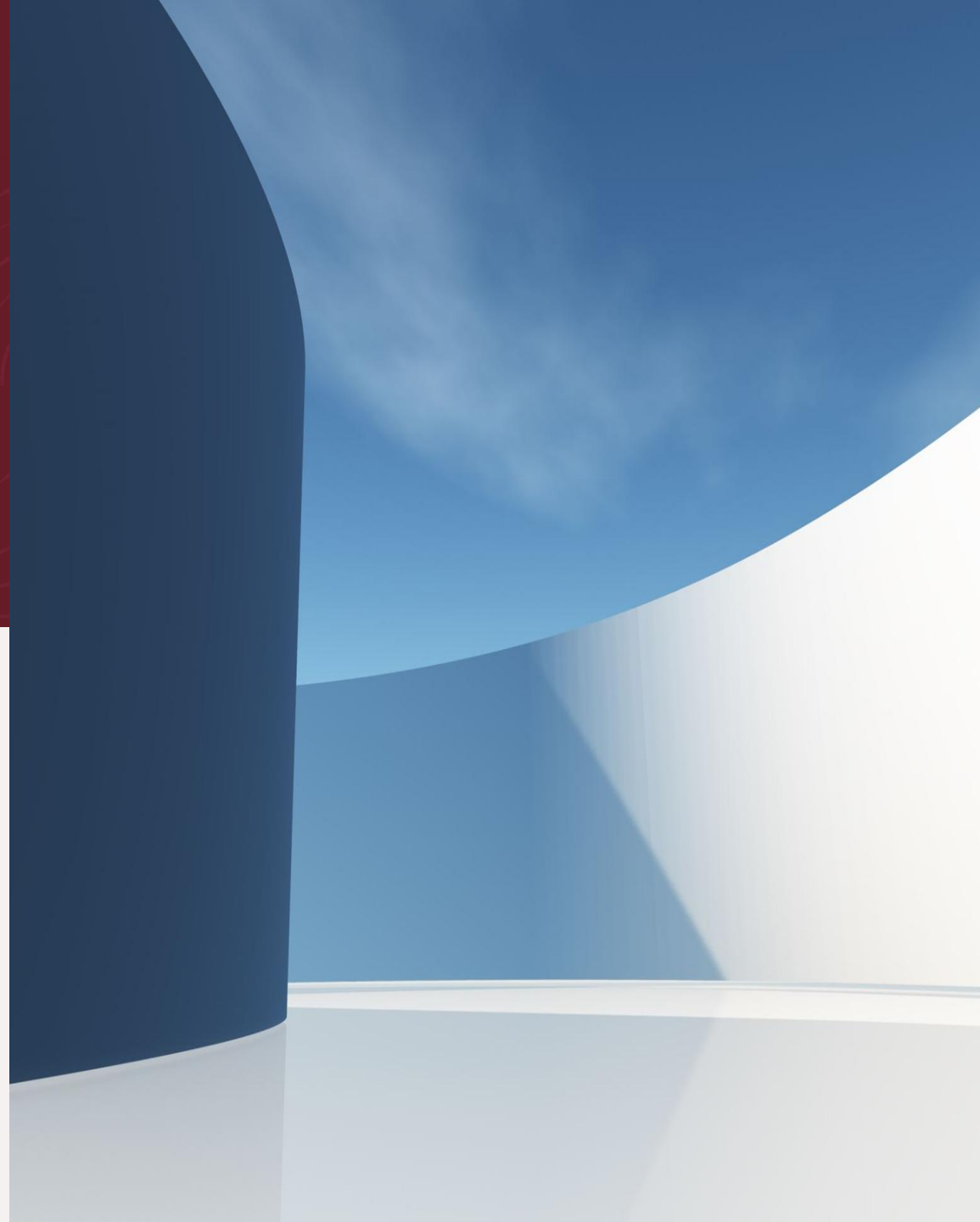
AIML CA1
Assignment Part B

Name: Ng Yu Hang

Admin No: P2423500

Class: DAAA/FT/1B/07



Assignment Information

Objectives:

Create a supervised regression model to predict housing prices using the given datas

Background Information:

The dataset provided is to predict the housing prices in US based on various information, such as Region, House Size, Renovation status, etc.

Our Housing DataSet

Here we import our data using **pd.read_csv** and visualize our dataset in our notebook using **data.head(15)** which shows the first 15 columns

```
data = pd.read_csv('housing_price_data.csv')
data.head(15)
```

[163] ✓ 0.0s

	House ID	City	House Area (sqm)	No. of Bedrooms	No. of Toilets	Stories	Renovation Status	Price ($)
0	0	Chicago	742.00	4	2	3	furnished	1330000
1	1	Denver	896.00	4	4	4	furnished	1225000
2	2	Chicago	996.00	3	2	2	semi-furnished	1225000
3	3	Seattle	750.00	4	2	2	furnished	1221500
4	4	New York	742.00	4	1	2	furnished	1141000
5	5	Boston	750.00	3	3	1	semi-furnished	1085000
6	6	Denver	858.00	4	3	4	semi-furnished	1015000
7	7	New York	1620.00	5	3	2	unfurnished	1015000
8	8	Denver	810.00	4	1	2	furnished	987000
9	9	Seattle	575.00	3	2	4	unfurnished	980000
10	10	Boston	1320.00	3	1	2	furnished	980000
11	11	Boston	600.00	4	3	2	semi-furnished	968100
12	12	Seattle	655.00	4	2	2	semi-furnished	931000
13	13	Boston	350.00	4	2	2	furnished	924000
14	14	Chicago	780.00	3	2	2	semi-furnished	924000

What are we working with?

```
data.describe().style.background_gradient(cmap="Purples")
```

	House ID	House Area (sqm)	No. of Bedrooms	No. of Toilets	Stories	Price ($)
count	545.000000	545.000000	545.000000	545.000000	545.000000	545.000000
mean	272.000000	515.054128	2.965138	1.286239	1.805505	476672.924771
std	157.472220	217.014102	0.738064	0.502470	0.867492	187043.961566
min	0.000000	165.000000	1.000000	1.000000	1.000000	175000.000000
25%	136.000000	360.000000	2.000000	1.000000	1.000000	343000.000000
50%	272.000000	460.000000	3.000000	1.000000	2.000000	434000.000000
75%	408.000000	636.000000	3.000000	2.000000	2.000000	574000.000000
max	544.000000	1620.000000	6.000000	4.000000	4.000000	1330000.000000

Using **data.describe()**, we can see what columns we have and the size of our dataset.

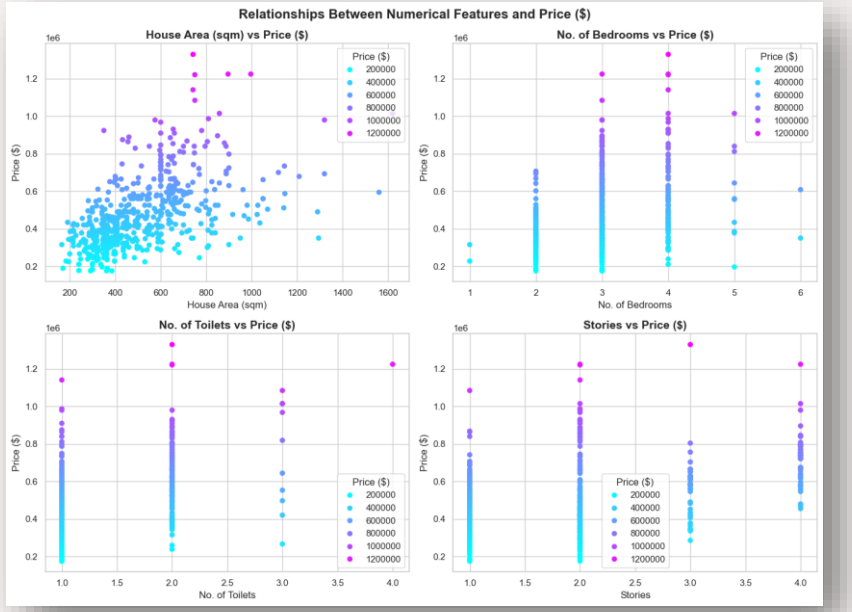
Using **data.isunull().sum()**, we get the total of missing datas for each column, which we can see from the result, there are none. Meaning, no imputation is required.

Check if there's missing data

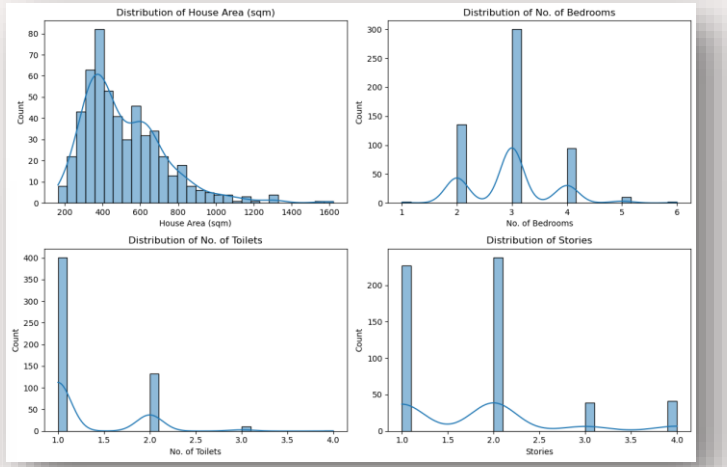
```
data.isnull().sum()
```

```
House ID              0
City                  0
House Area (sqm)      0
No. of Bedrooms       0
No. of Toilets        0
Stories               0
Renovation Status     0
Price ($)             0
dtype: int64
```

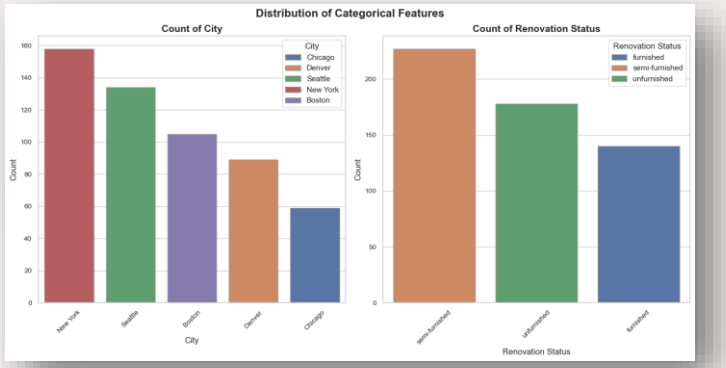
Data Exploration



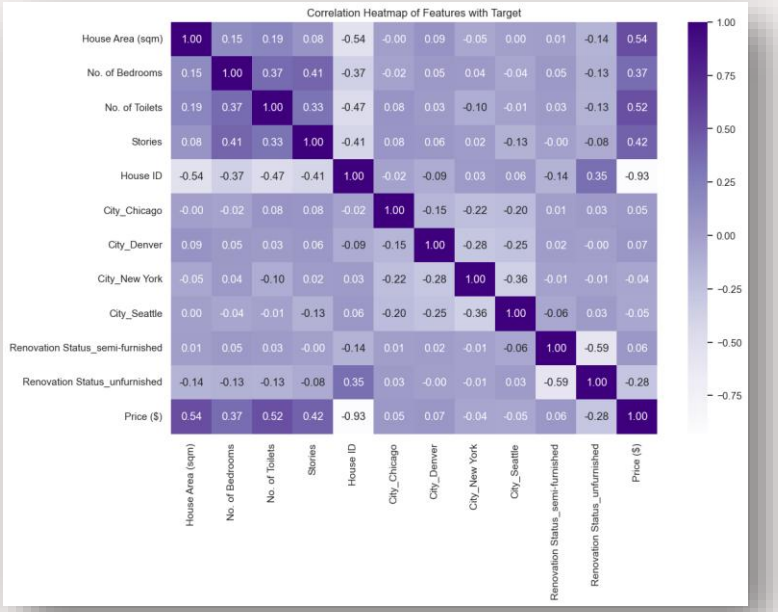
Scatter plot to show relationship between features



Bar graph – Distribution of numerical features



Bar graph - Distribution of categorical features



Correlation heatmap to show correlation of each features to target.

Feature Engineering

```
data['Bedroom_to_Area_Ratio'] = data['No. of Bedrooms'] / data['House Area (sqm)']
data['Toilet_to_Area_Ratio'] = data['No. of Toilets'] / data['House Area (sqm)']
data['Total_Livable_Area'] = data['House Area (sqm)'] * data['Stories']
renovation_map = {'unfurnished': 0, 'semi-furnished': 1, 'furnished': 2}
data['Encoded_Renovation_Status'] = data['Renovation Status'].map(renovation_map)
data['Bedrooms_per_Toilet'] = data['No. of Bedrooms'] / data['No. of Toilets']


correlation_feature_columns = [
    'Bedroom_to_Area_Ratio', 'Toilet_to_Area_Ratio',
    'Total_Livable_Area', 'Encoded_Renovation_Status', 'Bedrooms_per_Toilet'
]

plt.figure(figsize=(14, 10))
newcorrelation_matrix = data[correlation_feature_columns + ['Price ($)']].corr()
sns.heatmap(newcorrelation_matrix, annot=True, cmap='Purples', fmt='.2f', linewidths=0.5)
plt.title('Correlation Heatmap with City-Specific Price per Square Meter Features', fontsize=16)
plt.show()
```

Plotting our new features on a correlation heatmap


```
numerical_features  = numerical_features + [
    'Encoded_Renovation_Status', 'Bedroom_to_Area_Ratio',
    'Total_Livable_Area', 'Bedrooms_per_Toilet'
]

feature_columns = numerical_features + categorical_features
```

✓ 0.0s

Adding the new features to our numerical features variable

Here, we **add new features** by defining new columns from existing datas based on our research to try to help the models learn better. By defining new columns in our data, using existing data, we **feature engineer new datas**. Then, using a **correlation heatmap matrix**, I plot the new features with the target (price ($)) to see how much they correlate.

Data preparation

Data Preparation

```
target = 'Price ($)'
y = data[target]
X = data[feature_columns]
X.head()
```

✓ 0.0s

	House Area (sqm)	No. of Bedrooms	No. of Toilets	Stories	Encoded_Renovation_Status	Bedroom_to_Area_Ratio	Total_Livable_Area	Bedrooms_per_Toilet	City	Renovation Status
0	742.0	4	2	3	2	0.005391	2226.0	2.0	Chicago	furnished
1	896.0	4	4	4	2	0.004464	3584.0	1.0	Denver	furnished
2	996.0	3	2	2	1	0.003012	1992.0	1.5	Chicago	semi-furnished
3	750.0	4	2	2	2	0.005333	1500.0	2.0	Seattle	furnished
4	742.0	4	1	2	2	0.005391	1484.0	4.0	New York	furnished

Here, we identify the target as the column **"Price ($)"** as we want to predict the housing prices. We set this to be the y data, and the X data to be the features, which now includes our newly created features

Train-Test Split data

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

print("Training set size:", X_train.shape)
print("Testing set size:", X_test.shape)

X_train = X_train.reset_index(drop=True)
X_test = X_test.reset_index(drop=True)
y_train = y_train.reset_index(drop=True)
y_test = y_test.reset_index(drop=True)
```

✓ 0.0s

```
Training set size: (381, 10)
Testing set size: (164, 10)
```

Then, we split the datas using **train_test_split** to split our training data and testing data into 70% and 30% respectively. This is so that we have sufficient data to test our trained model on unknown data to provide a more reliable performance result.

Preprocessing

```
numerical_preprocessor = Pipeline([
    ('scaler', StandardScaler())
])

categorical_preprocessor = Pipeline([
    ('onehot', OneHotEncoder(drop='first'))
])

preprocessor = ColumnTransformer([
    ('num', numerical_preprocessor, numerical_features),
    ('cat', categorical_preprocessor, categorical_features)
])
```

✓ 0.0s

Using **StandardScaler()** and **OneHotEncoder()** to standardize and transform our data into those suitable for model learning

Setting up model-tuning components

Setting the hyperparameters to tune our models

```
param_grids = {
    'Linear Regression': {},

    'Ridge': {
        'ridge__alpha': [0.1, 1.0, 10.0],
        'ridge__solver': ['auto', 'svd', 'saga']
    },

    'Lasso': {
        'lasso__alpha': [0.01, 0.1, 1.0],
        'lasso__max_iter': [5000, 10000, 20000]
    },

    'Decision Tree': {
        'dt__max_depth': [6, 10, None],
        'dt__min_samples_split': [2, 5, 10],
        'dt__min_samples_leaf': [1, 3, 5]
    },

    'Random Forest': {
        'forest__n_estimators': [50, 100, 150],
        'forest__max_depth': [10, 15, None],
        'forest__min_samples_split': [2, 5, 10]
    },

    'Gradient Boosting': {
        'gb__n_estimators': [100, 150, 200],
        'gb__learning_rate': [0.03, 0.05, 0.1],
        'gb__max_depth': [3, 5, 7]
    },

    'AdaBoost': {
        'ab__n_estimators': [50, 100, 150],
        'ab__learning_rate': [0.03, 0.1, 0.2]
    },

    'SVR': {
        'svr__C': [1.0, 10.0, 50.0],
        'svr__epsilon': [0.01, 0.1, 0.5],
        'svr__kernel': ['linear', 'rbf']
    }
}
```

Setting up our KFolds

```
kf = KFold(n_splits=5, shuffle=True, random_state=42)
```

✓ 0.0s

Here, we set up our **hyperparameter tuning parameters** in our parameter grid to be used later with the models. (**balanced so it can catch patterns while not overfitting to outliers**)

We also setup our **Kfold** as our **Cross Validation Method** to enhance the reliability of our model's evaluation results by providing better estimates on unseen data.

Setting up our models

Defining our models with pipeline to include our hyperparameter tuning and preprocessors

```
models = {
    'Linear Regression': Pipeline([
        ('preprocessor', preprocessor),
        ('linear', LinearRegression())
    ]),
    'Ridge': Pipeline([
        ('preprocessor', preprocessor),
        ('ridge', Ridge())
    ]),
    'Lasso': Pipeline([
        ('preprocessor', preprocessor),
        ('lasso', Lasso(alpha=10, max_iter=200000))
    ]),
    'Decision Tree': Pipeline([
        ('preprocessor', preprocessor),
        ('dt', DecisionTreeRegressor())
    ]),
    'Random Forest': Pipeline([
        ('preprocessor', preprocessor),
        ('forest', RandomForestRegressor(random_state=42))
    ]),
    'Gradient Boosting': Pipeline([
        ('preprocessor', preprocessor),
        ('gb', GradientBoostingRegressor(random_state=42))
    ]),
    'AdaBoost': Pipeline([
        ('preprocessor', preprocessor),
        ('ab', AdaBoostRegressor(random_state=42))
    ]),
    'SVR': Pipeline([
        ('preprocessor', preprocessor),
        ('svr', SVR())
    ])
}
```

Here, we define our models using a **pipeline**, which helps organize neatly to implement our predefined **preprocessors** to our training data each time before training on the **models.**
This helps to ensure **consistency** and also helps **making changes easier** and **less prone to errors.**

Feature Importance

Feature Importance of models that support it

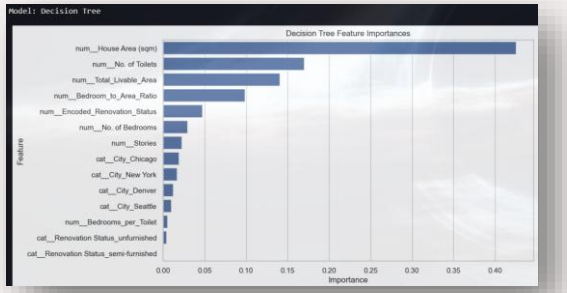
```python
for model_name, pipeline in models.items():
    pipeline.fit(X_train, y_train)
    best_model = pipeline
    preprocessor = best_model.named_steps.get('preprocessor', None)
    model_step_name = list(best_model.named_steps.keys())[-1]
    model = best_model.named_steps[model_step_name]

    # Check if the model supports feature_importances_
    if hasattr(model, 'feature_importances_'):
        print(f"Model: {model_name}")
        feature_importances = model.feature_importances_
        feature_names = preprocessor.get_feature_names_out()
        feature_importance_df = pd.DataFrame({
            'Feature': feature_names,
            'Importance': feature_importances
        }).sort_values(by='Importance', ascending=False)

        # Plot feature importances
        plt.figure(figsize=(10, 6))
        sns.barplot(x='Importance', y='Feature', data=feature_importance_df)
        plt.title(f'{model_name} Feature Importances')
        plt.show()
```

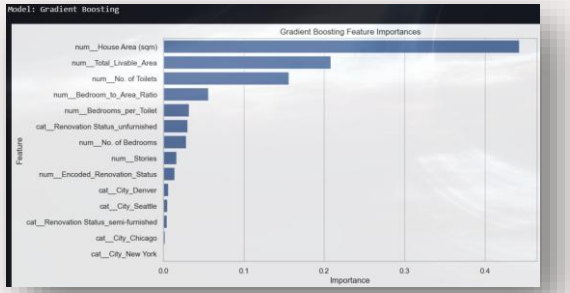
[20] ✓ 1.1s

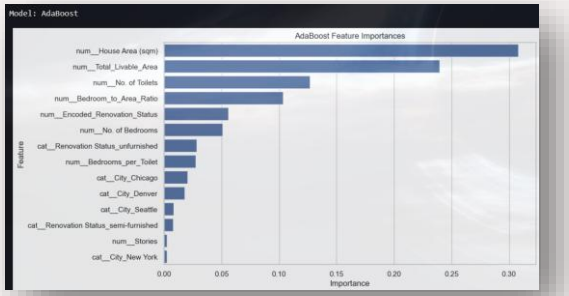
We now train the base models without hyperparameter tuning because **Feature Importance is model-specific.** And tuning generally **do not change the feature importances much.**

For models that support it (i.e. have the attribute **feature_importances_**), we show the name of the model and their respective feature importances of those models by plotting a bar plot.

Example outputs







Running the models and evaluate them

Running the models

```
results = {}

for model_name, pipeline in models.items():
    print(f"Training {model_name}...")

    # Perform grid search
    grid = GridSearchCV(pipeline, param_grid=param_grids[model_name], cv=kf, scoring='neg_mean_squared_error', verbose=2)
    grid.fit(X_train, y_train)

    # Best model
    best_model = grid.best_estimator_
    y_pred = best_model.predict(X_test)

    # Evaluate performance
    mae = mean_absolute_error(y_test, y_pred)
    mse = mean_squared_error(y_test, y_pred)
    r2 = r2_score(y_test, y_pred)
    mape = np.mean(np.abs((y_test - y_pred) / y_test)) * 100

    results[model_name] = {
        'Best Parameters': grid.best_params_,
        'MAE': mae,
        'MSE': mse,
        'R²': r2,
        'MAPE': mape
    }
```

Now, we run our models by using **GridSearchCV**, using our **pipeline, hyperparameter param_grid and Kfolds**, we can get the **average performance** of all models with **each hyparameter** across **all folds.**

We then convert that result into a dataframe, highlighting **green** for the **best** performance, and **red** for the **worst** performance. From the result, we can see we have **Ridge** as our best model in that output.

Performance Result

```
results_df = pd.DataFrame(results).T

results_df = results_df.style.apply(
    lambda x: [
        "background-color: green; color: white" if v == x.min() else
        "background-color: red; color: white" if v == x.max() else ""
        for v in x
    ],
    subset=['MAE', 'MSE', 'MAPE']
).apply(
    lambda x: [
        "background-color: green; color: white" if v == x.max() else
        "background-color: red; color: white" if v == x.min() else ""
        for v in x
    ],
    subset=['R²']
)

print("Model Evaluation Results:\n")
print("Legend: \nGreen - Best Performance; Red - Worst Performance")
results_df
```

Example output

	Best Parameters	MAE	MSE	R^2	MAPE
Linear Regression	{}	107461.711389	20802324776.557007	0.516943	25.072803
Ridge	{'ridge__alpha': 10.0, 'ridge__solver': 'saga'}	106747.068492	20489893424.235493	0.524198	24.895456
Lasso	{'lasso__alpha': 1.0, 'lasso__max_iter': 5000}	107459.957764	20801775408.507088	0.516956	25.072436
Decision Tree	{'dt__max_depth': 6, 'dt__min_samples_leaf': 5, 'dt__min_samples_split': 5}	108373.900655	22216616604.062397	0.484101	25.352562
Random Forest	{'forest__max_depth': 10, 'forest__min_samples_split': 10, 'forest__n_estimators': 100}	106367.971448	21352094295.873825	0.504177	24.802835
Gradient Boosting	{'gb__learning_rate': 0.03, 'gb__max_depth': 3, 'gb__n_estimators': 150}	107659.263578	21273643419.498463	0.505998	25.398451
AdaBoost	{'ab__learning_rate': 0.03, 'ab__n_estimators': 150}	114049.023620	22634827830.904278	0.474390	27.638326
SVR	{'svr__C': 50.0, 'svr__epsilon': 0.01, 'svr__kernel': 'linear'}	141807.217940	35706922472.160217	0.170839	32.447890

Setting up model with important features only

```
columns_to_drop = ['City', 'Renovation Status', 'Stories']

# Update feature_columns by removing the specified columns
newfeature_columns = [feature for feature in feature_columns if feature not in columns_to_drop]
newfeature_columns
```

```
['House Area (sqm)',
 'No. of Bedrooms',
 'No. of Toilets',
 'Encoded_Renovation_Status',
 'Bedroom_to_Area_Ratio',
 'Total_Livable_Area',
 'Bedrooms_per_Toilet']
```

Removing unimportant features

Removing the unnecessary features from what the feature importance showed

```
tuned_X = data[newfeature_columns]

tuned_X_train, tuned_X_test, tuned_y_train, tuned_y_test = train_test_split(tuned_X, y, test_size=0.3, random_state=42)

print("Training set size:", X_train.shape)
print("Testing set size:", X_test.shape)

tuned_X_train = tuned_X_train.reset_index(drop=True)
tuned_X_test = tuned_X_test.reset_index(drop=True)
tuned_y_train = tuned_y_train.reset_index(drop=True)
tuned_y_test = tuned_y_test.reset_index(drop=True)
```

```
Training set size: (381, 10)
Testing set size: (164, 10)
```

Setting up X and y with the new dataset

From the **feature importance bar plot**, we saw a few features that most models find **unimportant**, so I remove it and retrained with the **best hyperparameters** from the previous model results to see if there's an **improvement** in model performance.

Defining our new models with the best parameters

```
newpreprocessor = ColumnTransformer([
    ('num', numerical_preprocessor, newfeature_columns)
])
```

There are no more categorical features after we removed cities, so we can just use the feature columns for preprocessing

```
newparam_grids = {
    'Linear Regression': {},
    'Ridge': {
        'ridge__alpha': [10.0]
        },
    'Random Forest': {'forest__max_depth': [5],
                      'forest__min_samples_split': [10],
                      'forest__n_estimators': [200]
                      },
    'Lasso': {
        'lasso__alpha': [1.0]
        }
}
```

```
best_models = {
    'Linear Regression': Pipeline([
        ('preprocessor', newpreprocessor),
        ('linear', LinearRegression())
    ]),
    'Ridge': Pipeline([
        ('preprocessor', newpreprocessor),
        ('ridge', Ridge())
    ]),
    'Random Forest': Pipeline([
        ('preprocessor', newpreprocessor),
        ('forest', RandomForestRegressor(random_state=42))
    ]),
    'Lasso': Pipeline([
        ('preprocessor', newpreprocessor),
        ('lasso', Lasso())
    ])
}
```

Using the best hyperparameters for the models

Running and evaluating our new models

Training our models with the best hyperparameters

```
tuned_results = {}

for model_name, pipeline in best_models.items():
    print(f"\nTraining and evaluating model: {model_name}\n")

    cv_scores = cross_val_score(
        pipeline,
        tuned_X_train,
        tuned_y_train,
        cv=kf,
        scoring='neg_mean_squared_error',
        verbose=3
    )

    pipeline.fit(tuned_X_train, tuned_y_train)
    tuned_y_pred = pipeline.predict(tuned_X_test)

    mae = mean_absolute_error(tuned_y_test, tuned_y_pred)
    mse = mean_squared_error(tuned_y_test, tuned_y_pred)
    r2 = r2_score(tuned_y_test, tuned_y_pred)
    mape = np.mean(np.abs((tuned_y_test - tuned_y_pred) / tuned_y_test)) * 100

    tuned_results[model_name] = {
        'MAE': mae,
        'MSE': mse,
        'R²': r2,
        'MAPE': mape
    }
```

Display Results of our tuned models

```
tuned_results_df = pd.DataFrame(tuned_results).T

tuned_results_df = tuned_results_df.style.apply(
    lambda x: [
        "background-color: green; color: white" if v == x.min() else
        "background-color: red; color: white" if v == x.max() else ""
        for v in x
    ],
    subset=['MAE', 'MSE', 'MAPE']
).apply(
    lambda x: [
        "background-color: green; color: white" if v == x.max() else
        "background-color: red; color: white" if v == x.min() else ""
        for v in x
    ],
    subset=['R²']
)

print("Model Evaluation Results:\n")
print("Legend: \nGreen - Best Performance; Red - Worst Performance")
tuned_results_df
```

Now, we train our models with the **best hyperparameters** as defined on the previous slide, with our new dataset consisting of only **features that were relatively important** to the models.

I then output the result in a **DataFrame**, and highlighting the **best performance** metrics once again.

```
Model Evaluation Results:

Legend:
Green - Best Performance; Red - Worst Performance
```

	MAE	MSE	R²	MAPE
Linear Regression	106478.194817	20562362632.338398	0.522515	25.008845
Ridge	106447.500791	20525450480.007473	0.523372	24.991854
Random Forest	109459.561606	22613652902.651436	0.474882	25.309923

Feature Removal Conclusion

Full Feature VS Removed Feature

+ Code

```
results_df = results_df.drop(columns=["Best Parameters"], errors='ignore')

models_to_compare = tuned_results.keys()
full_features_filtered = results_df.loc[models_to_compare]
reduced_features_filtered = tuned_results_df

comparison_df = full_features_filtered - reduced_features_filtered


comparison_df = comparison_df.style.apply(
    lambda x: [
        "background-color: green; color: white" if v > 0 else
        "background-color: red; color: white"
        for v in x
    ],
    subset=['MAE', 'MSE', 'MAPE']
).apply(
    lambda x: [
        "background-color: green; color: white" if v < 0 else
        "background-color: red; color: white"
        for v in x
    ],
    subset=['R²']
)

print("Green = improvement, red = decreased performance")

comparison_df
```

Comparing all features vs removed features metrics

Output

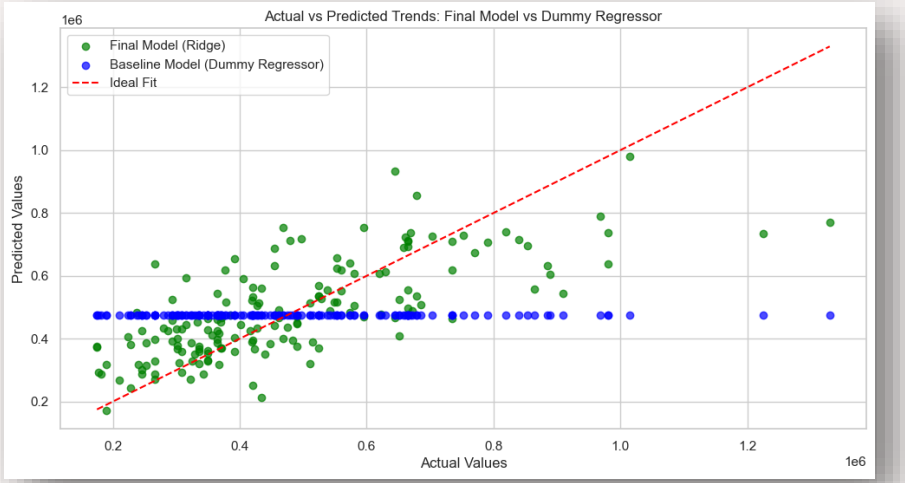
```
Green = improvement, red = decreased performance
```

	MAE	MSE	R^2	MAPE
Linear Regression	983.516572	239962144.218609	-0.005572	0.063958
Ridge	299.567701	-35557055.771980	0.000826	-0.096398
Random Forest	-3091.590158	-1261558606.777611	0.029295	-0.507088

Conclusion: Comparing the metrics of the removed features vs the all features models, we can conclude that removing the features **DID NOT** cause an overall performance improvement for the models. Hence, we will **keep the previous features** as it was.

Comparing our best model to a baseline model

```python
# Fit a Dummy Regressor
dummy_model = DummyRegressor(strategy="mean")
dummy_model.fit(X_train, y_train)
dummy_predictions = dummy_model.predict(X_test)

# Final model predictions
final_model = best_models['Ridge']
final_predictions = final_model.predict(X_test)

# Plot Actual vs Predicted Trends
plt.figure(figsize=(12, 6))

plt.scatter(y_test, final_predictions, label="Final Model (Ridge)", alpha=0.7, color='green')
plt.scatter(y_test, dummy_predictions, label="Baseline Model (Dummy Regressor)", alpha=0.7, color='blue')
plt.plot([min(y_test), max(y_test)], [min(y_test), max(y_test)],
         color='red', linestyle='--', label="Ideal Fit")

plt.title("Actual vs Predicted Trends: Final Model vs Dummy Regressor")
plt.xlabel("Actual Values")
plt.ylabel("Predicted Values")
plt.legend()
plt.grid(True)
plt.show()
```



Here, we train a quick dummy model using **dummyregressor()** to act as a **baseline**. Then we plot a scatter graph of **predicted values** vs **actual values** to see how accurate the models are.

From the graph, our model is **better** at predicting close to the actual values than the dummy.

Concluding: Best Model vs Baseline Model

```python
mae = mean_absolute_error(y_test, dummy_predictions)
mse = mean_squared_error(y_test, dummy_predictions)
r2 = r2_score(y_test, dummy_predictions)
mape = np.mean(np.abs((y_test - dummy_predictions) / y_test)) * 100

final_mae = mean_absolute_error(y_test, final_predictions)
final_mse = mean_squared_error(y_test, final_predictions)
final_r2 = r2_score(y_test, final_predictions)
final_mape = np.mean(np.abs((y_test - final_predictions) / y_test)) * 100

dummy_results = {
    'MAE': mae,
    'MSE': mse,
    'R²': r2,
    'MAPE': mape,
}

final_results = {
    'MAE': final_mae,
    'MSE': final_mse,
    'R²': final_r2,
    'MAPE': final_mape,
}

dummy_results_df = pd.DataFrame(dummy_results, index=["Dummy Model"]).T
dummy_results_df.columns = ["Dummy Model"]

combined_results = pd.DataFrame({
    "Dummy Model": dummy_results,
    "Final Model (Ridge)": final_results
}).T

combined_results_df = combined_results.style.apply(
    lambda x: [
        "background-color: green; color: white" if v == x.min() else
        "background-color: red; color: white" if v == x.max() else ""
        for v in x
    ],
    subset=['MAE', 'MSE', 'MAPE']
).apply(
    lambda x: [
        "background-color: green; color: white" if v == x.max() else
        "background-color: red; color: white" if v == x.min() else ""
        for v in x
    ],
    subset=['R²']
)
combined_results_df
```

	MAE	MSE	R^2	MAPE
Dummy Model	161771.174173	43065975934.607613	-0.000048	39.536136
Final Model (Ridge)	106447.500791	20525450480.007473	0.523372	24.991854

To better compare them, we can compare their scores by displaying the metric scores for both the **best model** and the **dummy model.**

From the table, we can see that our **Final Model** is **significantly** better than our dummy model, hence proving **our model is significantly better than the model baseline**

Final Check (Reliability) : Under/overfitting?

```python
ridge_model = models['Ridge']
best_params = newparam_grids['Ridge']

ridge_model.set_params(**best_params)
ridge_model.fit(X_train, y_train)


train_predictions = ridge_model.predict(X_train)
test_predictions = ridge_model.predict(X_test)
train_r2 = r2_score(y_train, train_predictions)
test_r2 = r2_score(y_test, test_predictions)

train_test_results = {
    'Train R²': train_r2,
    'Test R²': test_r2
}
overfitting = train_r2 - test_r2

print(f"Model: Ridge")
print(f"Train R²: {train_r2:.4f}")
print(f"Test R²: {test_r2:.4f}\n")
print(f"Overfitting test (Train R² - Test R²): {overfitting:.4f}\n")

if overfitting > 0.1:
    print("Conclusion: This model might be overfitting.")
elif train_r2 < 0.3 and test_r2 < 0.3:
    print("Conclusion: This model might be underfitting.")
else:
    print("Conclusion: No overfitting or underfitting detected. The model generalizes well.")
```

```
Model: Ridge
Train R²: 0.5960
Test R²: 0.5242

Overfitting test (Train R² - Test R²): 0.0718

Conclusion: No overfitting or underfitting detected. The model generalizes well.
```

As our final check before we conclude the best model, we want to check if it is **reliable** to be used.

We can check if it is **underfitting** or **overfitting** by comparing the **train R2 score** with the **test R2 score**. If there is **not** a **significant difference**, it shows the model is good at capturing the pattern and is reliable.

Final Conclusion

	Best Parameters	MAE	MSE	R^2	MAPE
Linear Regression	{}	107461.711389	20802324776.557007	0.516943	25.072803
Ridge	{'ridge__alpha': 10.0, 'ridge__solver': 'saga'}	106747.068492	20489893424.235493	0.524198	24.895456
Lasso	{'lasso__alpha': 1.0, 'lasso__max_iter': 5000}	107459.957764	20801775408.507088	0.516956	25.072436
Decision Tree	{'dt__max_depth': 6, 'dt__min_samples_leaf': 5, 'dt__min_samples_split': 5}	108373.900655	22216616604.062397	0.484101	25.352562
Random Forest	{'forest__max_depth': 10, 'forest__min_samples_split': 10, 'forest__n_estimators': 100}	106367.971448	21352094295.873825	0.504177	24.802835
Gradient Boosting	{'gb__learning_rate': 0.03, 'gb__max_depth': 3, 'gb__n_estimators': 150}	107659.263578	21273643419.498463	0.505998	25.398451
AdaBoost	{'ab__learning_rate': 0.03, 'ab__n_estimators': 150}	114049.023620	22634827830.904278	0.474390	27.638326
SVR	{'svr__C': 50.0, 'svr__epsilon': 0.01, 'svr__kernel': 'linear'}	141807.217940	35706922472.160217	0.170839	32.447890

	MAE	MSE	R^2	MAPE
Dummy Model	161771.174173	43065975934.607613	-0.000048	39.536136
Final Model (Ridge)	106447.500791	20525450480.007473	0.523372	24.991854

As previously compared, our **best** model is **Ridge Regression**, with the best overall performance in every metric-wise and reliable to use.

By previous comparison between removing and not removing features, we can conclude the best model and features for this dataset according to our tests.

In conclusion, the best model for this task is Ridge Regression, with all the features given in the original dataset + my own feature engineered features.